

Name: Soumyadip Ghosh

Registration No.: 23BCB0067

Course: Web Programming (BCSE203E)

Topic: Visual SQL Builder

**Review 2 : Module/Functions Design and
Implementation document**

1. Introduction

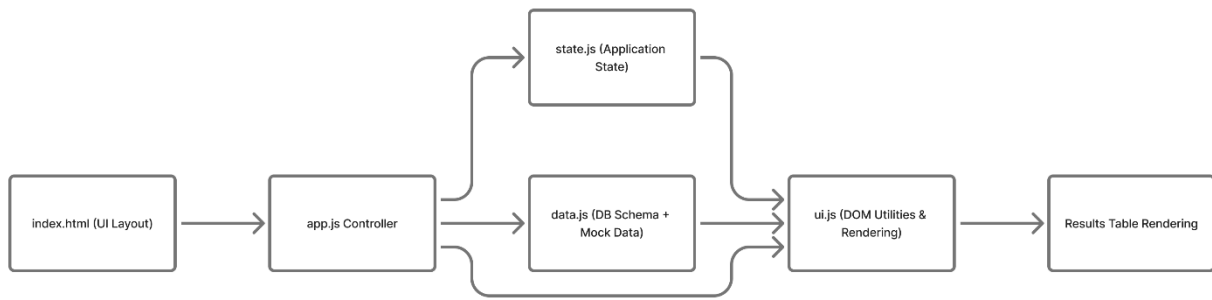
The **Visual SQL Query Builder** is a client-side web application developed using **HTML, CSS, and JavaScript**. The application enables users to visually construct SQL queries without manually writing SQL syntax.

The system provides an intuitive interface where users can:

- Select database tables
- Choose columns to retrieve
- Apply filtering conditions
- Sort query results
- Limit returned records

Since the system operates entirely on the client side, database operations are simulated using **mock datasets stored in JavaScript modules**. The application follows a **modular architecture and Model–View–Controller (MVC) design pattern** to ensure maintainability and scalability.

2. Module Architecture Diagram



The system is divided into multiple modules, each responsible for a specific functionality. This modular structure improves code organization, reusability, and maintainability.

The main modules of the system are:

2.1 HTML Interface (`index.html`)

This file defines the **structure of the application interface**. It includes components such as:

- Query builder panel
- Table selection dropdown
- Column selection checkboxes
- Filter inputs
- Results display table

It serves as the **entry point of the application**.

2.2 Controller Module (`app.js`)

The **controller module** manages the interaction between the user interface and application data.

Responsibilities include:

- Listening to user events
- Updating application state

- Generating SQL queries
- Triggering result rendering

It acts as the **central coordinator of the system**.

2.3 State Management Module (state.js)

This module stores the **current state of the query being constructed**.

The state includes:

- Selected table
- Selected columns
- Filter conditions
- Sorting options
- Result limit

Maintaining a centralized state allows the application to **track user input consistently across different components**.

2.4 Data Module (data.js)

The data module simulates a **database environment**.

It contains:

- Database schema definitions
- Mock datasets representing table records

This module allows the system to **simulate SQL query execution without requiring a real database connection**.

2.5 UI Rendering Module (ui.js)

The UI module handles **DOM manipulation and dynamic interface updates**.

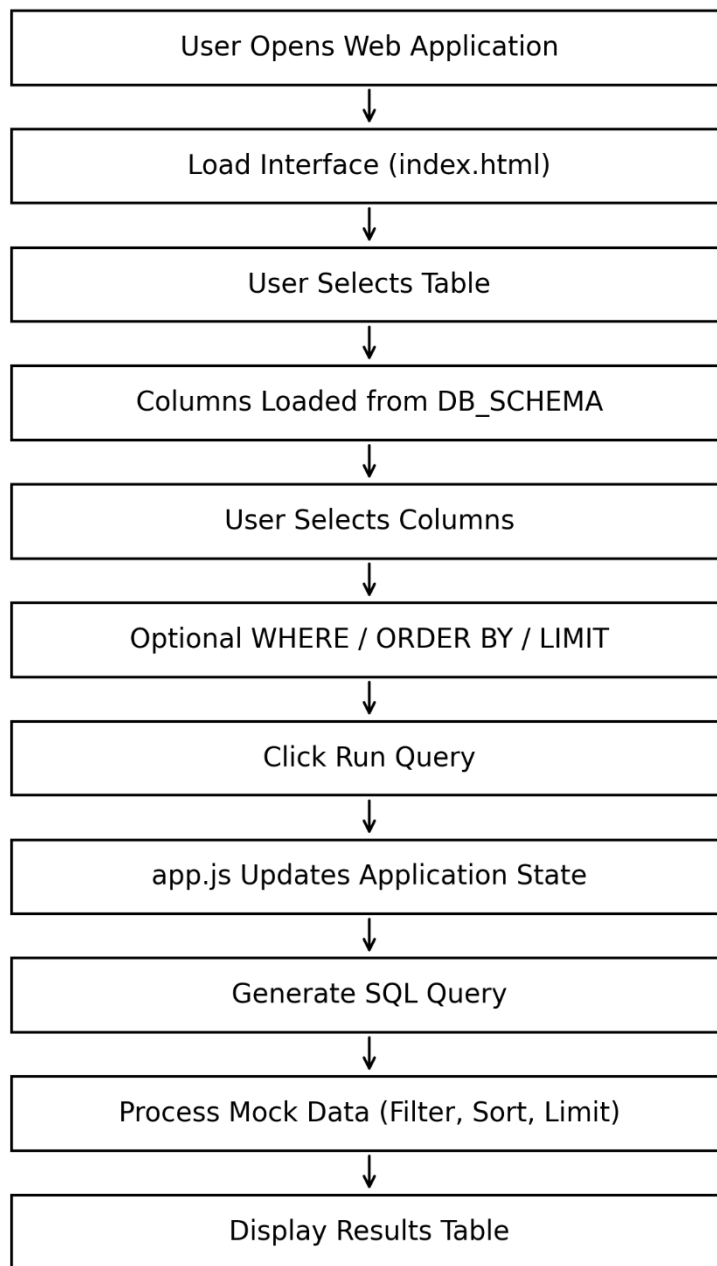
Functions in this module perform tasks such as:

- Accessing HTML elements
- Populating dropdown menus

- Rendering query results
- Updating the results table dynamically

This module ensures the interface updates correctly based on user actions.

3. Application Flow Diagram



The Application Flow Diagram illustrates how data flows through the system when a user builds and executes a SQL query.

Step 1: Application Initialization

When the user opens the website, the HTML interface loads and the JavaScript controller initializes event listeners.

Step 2: Table Selection

The user selects a table from the **FROM dropdown menu**.

The application then:

- Retrieves the schema of the selected table
- Dynamically generates column selection checkboxes
- Updates filter and sorting dropdown menus

Step 3: Column Selection

The user selects one or more columns to retrieve. These selections form the **SELECT clause** of the SQL query.

The selected columns are stored in the **application state**.

Step 4: Query Configuration

The user may optionally specify:

- **WHERE clause** to filter records
- **ORDER BY clause** to sort results
- **LIMIT clause** to restrict number of rows

These parameters are stored in the state object.

Step 5: Query Execution

When the **Run Query** button is pressed, the application:

1. Validates user input
2. Constructs a SQL query string
3. Updates the results display

Step 6: Query Processing

The system processes the query using JavaScript array operations:

- Filtering records
- Sorting results

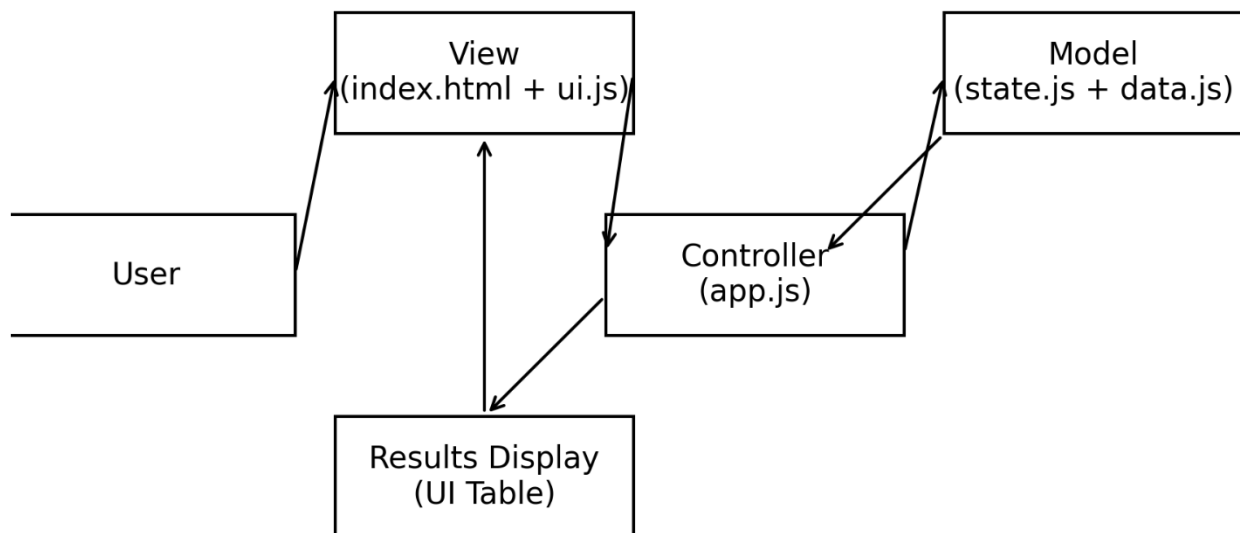
- Limiting the dataset

This simulates the behavior of an actual SQL database.

Step 7: Results Display

The processed data is rendered dynamically in the results table. The generated SQL query is also displayed for reference.

4. MVC Architecture Diagram



The application follows the **Model–View–Controller (MVC)** architectural pattern to separate concerns and improve maintainability.

4.1 Model

The **Model** manages the data and application state.

Components:

- data.js
- state.js

Responsibilities:

- Store database schema
- Maintain mock data
- Manage current query state

The model represents the **logical data layer of the application**.

4.2 View

The **View** represents the user interface.

Components:

- index.html
- ui.js

Responsibilities:

- Display query builder interface
- Render query results
- Update UI elements dynamically

The view ensures that **user interactions are visually reflected on the interface**.

4.3 Controller

The **Controller** connects the Model and View.

Component:

- app.js

Responsibilities:

- Handle user interactions
- Update application state
- Trigger UI updates
- Coordinate query execution

The controller ensures smooth communication between the data layer and the user interface.